

SADRŽAJ

1. UVOD	7
2. PREGLED PODRUČJA	8
2.1. Povijest	8
2.2. Usporedba.....	11
2.2.1. Microsoftovi aplikacijski okviri	11
2.2.2. Nezavisne biblioteke i razvojna okruženja.....	11
3. FUNKCIONALNOST MFC-A.....	13
3.1. Windows programski model.....	13
3.2. Mapiranje poruka (Message Maps)	14
3.3. Hijerarhija klasa MFC-a	15
3.4. Dokument/pogled arhitektura	16
3.5. AFX funkcije	17
4. PODRŠKA U VISUAL STUDIJU	19
4.1. Integracija MFC-a u Visual Studio	19
4.2. MFC aplikacijski čarobnjak	19
4.3. Uređivači resursa	20
5. PRAKTIČAN RAD – UNO KARTAŠKA IGRA.....	21
5.1. Svrha aplikacije	21
5.2. Struktura aplikacije.....	22
5.3. Korištene MFC komponente	23
5.4. Korištenje GDI+ biblioteke	24
ZAKLJUČAK.....	26
LITERATURA	27
SAŽETAK	28
SUMMARY	29

POPIS SLIKA

Slika 1 MFC logo	8
Slika 2 Microsoft Office ribbon.....	9
Slika 3 Winmerge korisničko sučelje	11
Slika 4 Windows programski model	13
Slika 5 Hijerarhijsko stablo CObject klase.....	15
Slika 6 Prikaz dokument/pogled veze	17
Slika 7 Aplikacijski čarobnjak.....	19
Slika 8 Resource editor.....	20
Slika 9 Sučelje aplikacije Uno.....	21
Slika 10 Izgled GameSettings dijaloga.....	24

POPIS TABLICA

Tablica 1 Pregled verzija kroz godine	10
Tablica 2 Najčešće poruke.....	14
Tablica 3 Najčešće AFX funkcije.....	18

POPIS KODOVA

Kod 1 switch naredba unutar WndProc()	14
Kod 2 mapiranje poruke, ovime se zamjenjuje switch u WndProc() prikazan u kodu 1	15
Kod 4 Primjer deklariranja CButton objekta te mapiranja poruke	24
Kod 5 Primjer korištenja GDI+	25

1. UVOD

U suvremenom razvoju softvera, osobito u području izrade desktop aplikacija za Windows operacijski sustav, i dalje postoji velik broj rješenja temeljenih na tehnologijama koje su nastale prije modernih okvira poput .NET-a ili WinUI-a. Jedna od takvih tehnologija je Microsoft Foundation Classes (MFC), objektno-orijentirana C++ biblioteka koja pruža omotač (wrapper) oko Win32 API-ja i omogućuje izradu grafičkih korisničkih sučelja i upravljanje događajima na tradicionalan način. Unatoč svojoj starosti, MFC ostaje prisutan u brojnim industrijskim i naslijeđenim sustavima, gdje stabilnost, dugoročna održivost i kompatibilnost imaju prednost nad modernim vizualnim mogućnostima. Upravo zbog te trajnosti i praktične vrijednosti, MFC predstavlja zanimljiv predmet proučavanja, osobito u kontekstu analize postojećih aplikacija koje se i danas aktivno koriste.

Tema će se obraditi kombinacijom teorijskog pregleda i praktične analize. Teorijski dio uključuje opis MFC biblioteke, njezinu arhitekturu, ključne koncepte i ulogu u razvoju Windows aplikacija. Praktični dio usmjeren je na analizu razvijene aplikacije: njezinu strukturu, glavne funkcionalne cjeline, način komunikacije između komponenti te korištenje MFC mehanizama poput dijaloga, kontrola i upravljanja porukama. Posebna pozornost posvećuje se dijelovima programa koji su najvažniji za razumijevanje njegove namjene i načina rada.

Struktura rada organizirana je tako da najprije daje teorijski okvir potreban za razumijevanje tehnologije, zatim prikazuje analizu aplikacije, a na kraju donosi zaključke o prednostima i nedostacima korištenja MFC-a u suvremenom razvoju softvera. Takav pristup omogućuje da se tema obradi cjelovito: od općeg pregleda tehnologije, preko konkretne implementacije, do kritičkog razmatranja njezine aktualnosti i mogućih smjerova daljnjeg razvoja.

2. PREGLED PODRUČJA

2.1. Povijest

Microsoft Foundation Classes (MFC) predstavlja skup C++ klasa namijenjenih razvoju aplikacija za operacijski sustav Windows.[1][2] Nastao je početkom 1990-ih kao odgovor na sve veću složenost razvoja Windows aplikacija korištenjem Win32 API-ja. U to vrijeme programeri su morali ručno upravljati prozorima, porukama, kontrolama i memorijom kroz velik broj funkcija operacijskog sustava, što je razvoj činilo sporim i podložnim pogreškama.

Prva verzija MFC biblioteke predstavljena je 1992. godine zajedno s razvojnim okruženjem Microsoft C/C++ 7.0.[3] Osnovna ideja bila je primijeniti principe objektno orijentiranog programiranja na razvoj Windows aplikacija. Umjesto korištenja velikog broja Win32 funkcija, programeri su mogli koristiti C++ klase koje su predstavljale prozore, dijaloge, gumbe i druge elemente korisničkog sučelja. Time je razvoj postao jednostavniji, pregledniji i lakši za održavanje.

U ranim godinama razvoja MFC je bio poznat pod nazivom Application Framework Extensions (AFX), zbog čega se i danas u mnogim MFC projektima mogu pronaći nazivi datoteka, funkcija i makroa koji počinju prefiksom "Afx". Kasnije je biblioteka dobila današnji naziv Microsoft Foundation Classes, dok je prefiks ostao kao svojevrsni povijesni ostatak.



Slika 1 MFC logo

Izvor: Microsoft Corporation

Tijekom 1990-ih MFC je postao jedna od najvažnijih tehnologija za razvoj Windows desktop aplikacija. Njegova popularnost dodatno je porasla pojavom razvojnog okruženja Visual C++, koje je omogućilo grafičko stvaranje prozora, dijaloga i drugih

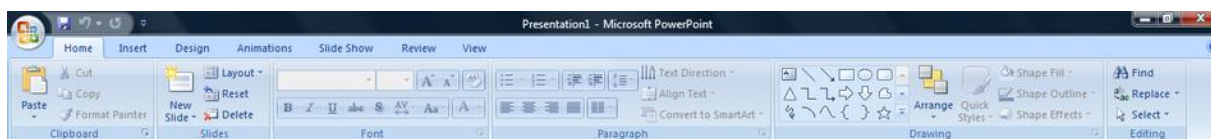
komponenti aplikacije. Posebno važna bila je integracija s Visual Studio okruženjem, koji nudi čarobnjake (Wizards) za automatsko stvaranje početne strukture programa. Na taj način programeri su mogli razvijati složene aplikacije bez potrebe za ručnim pisanjem cjelokupnog temeljnog koda.

Značajan doprinos popularnosti MFC-a dala je i implementacija dokument/pogled arhitekture. Ovaj koncept omogućio je odvajanje podataka aplikacije od njihovog prikaza na zaslonu, čime su aplikacije postale modularnije i jednostavnije za održavanje. Takav pristup bio je posebno koristan kod poslovnih aplikacija, uređivača dokumenata i tehničkih alata.

Krajem 1990-ih i početkom 2000-ih MFC je dosegao vrhunac popularnosti. Velik broj komercijalnih i industrijskih aplikacija razvijan je upravo pomoću ove biblioteke. U tom razdoblju nastali su brojni razvojni alati, CAD sustavi, znanstvene aplikacije te poslovna rješenja koja su se oslanjala na MFC i Win32 platformu. Mnoge od tih aplikacija koriste se i danas.

Početkom 21. stoljeća Microsoft uvodi .NET Framework, a kasnije i Windows Presentation Foundation (WPF). Te nove tehnologije donijele su moderniji pristup razvoju grafičkih korisničkih sučelja te su postupno preuzele ulogu glavnih razvojnih platformi za Windows aplikacije. Zbog toga je razvoj MFC-a usporen, iako biblioteka nije napuštena. Microsoft je nastavio pružati podršku te prilagođavati MFC novim verzijama Visual Studija i Windows operacijskog sustava.

Godine 2008. MFC je dobio značajnije proširenje koje je uključivalo podršku za moderne korisničke elemente kao što su ribbon sučelje, napredne alatne trake i poboljšane mogućnosti organizacije prozora. Cilj tih nadogradnji bio je omogućiti postojećim aplikacijama moderniji izgled bez potpune promjene tehnologije.



Slika 2 Microsoft Office ribbon

Danas se MFC smatra zrelom i stabilnom tehnologijom. Iako više nije prvi izbor za razvoj novih Windows aplikacija, i dalje je prisutan u velikom broju postojećih sustava. Razlozi za njegovu dugovječnost uključuju visoke performanse, izravnu povezanost s Win32 API-jem, stabilnost te činjenicu da mnoge organizacije održavaju aplikacije

razvijene tijekom posljednjih nekoliko desetljeća. Microsoft i dalje uključuje MFC u Visual Studio te omogućuje razvoj i održavanje postojećih projekata.

Tablica 1 Pregled verzija kroz godine

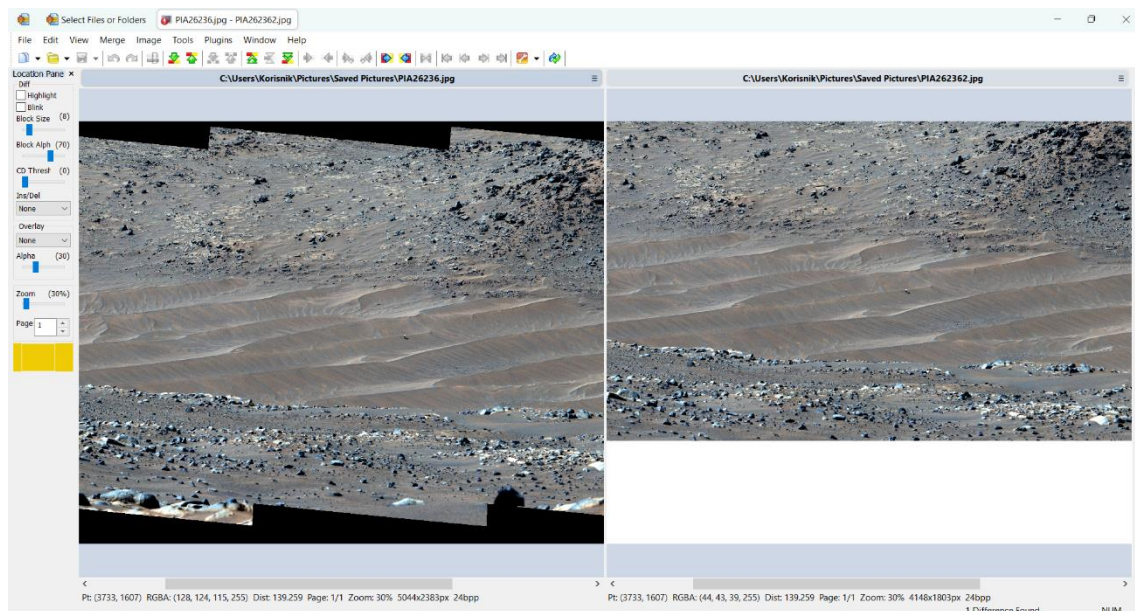
Godina	Verzija	Značajke
1992.	MFC 1.0	Prva verzija
1993.	MFC 2.0	Integracija s prvim verzijama Visual C++
1994.	MFC 3.x	Dodane nove klase i bolja integracija s 32-bitnim sustavima
1995.	MFC 4.0	Uvedena veća podrška za Win32 API
1996.	MFC 4.2	Dodana potpora za WinINet[
1998.	MFC 6.0	Dolazi uz Visual Studio 6.0, MFC je u ovom periodu bio najpopularniji
2002.	MFC 7.0	Integracija s .NET
2003.	MFC 7.1	
2005.	MFC 8.0	Dolazi uz Visual Studio 2005, poboljšana podrška za Windows sustave i sigurnosne mehanizme
2007.	MFC 9.0	Modernizacija sučelja, dodana podrška za ribbon, docking prozore te Office stil sučelja
2010. - danas	MFC 10.0 – MFC 14.x	Nema novih značajki, održava se kompatibilnost i sigurnosne nadogradnje. Uključen u Visual Studio 2026

Izvor [1][4]

2.2. Usporedba

Prednost MFC-a u odnosu na druge okvire jest izravan pristup Win32 API-ju, što osigurava visoke performanse uz manje dodatnih apstrakcija. Zbog toga se i danas koristi u pojedinim industrijskim i alatnim aplikacijama, poput programa WinMerge.[4]

Slika 3 Winmerge korisničko sučelje



Izvor: Slikao autor

2.2.1. Microsoftovi aplikacijski okviri

Osnovna ideja novih Microsoftovih okvira poput WPF (Windows Presentation Foundation) i WinUI 3 je odmaknuti programere od Win32 modela i pojednostaviti izradu sučelja

Za razliku od MFC-a, WPF i WinUI 3 koriste data binding i MVVM (Model-View-ViewModel) arhitekturni obrazac te XAML (Extensible Application Markup Language) baziran na XML-u za definiranje sučelja.

Razlika između WPF i WinUI 3 je to što je WPF baziran na .NET platformi, dok WinUI 3 koristi Windows App SDK i C#/C++. [5][6]

2.2.2. Nezavisne biblioteke i razvojna okruženja

Po arhitekturi najbliža je wxWidgets biblioteka, izvorno razvijena 1992. godine pod imenom wxWindows te poput MFC-a radi kao C++ omotač za izvorne API-je operacijskih sustava kao što su Windows, Linux i macOS te ograničena podrška za iOS.

Primjer wxWidgets aplikacija su Code::Blocks IDE za C/C++, a biblioteka se koristi se u industriji, znanosti i obrazovanju; među korisnicima su Xerox, AMD i NASA. [7]

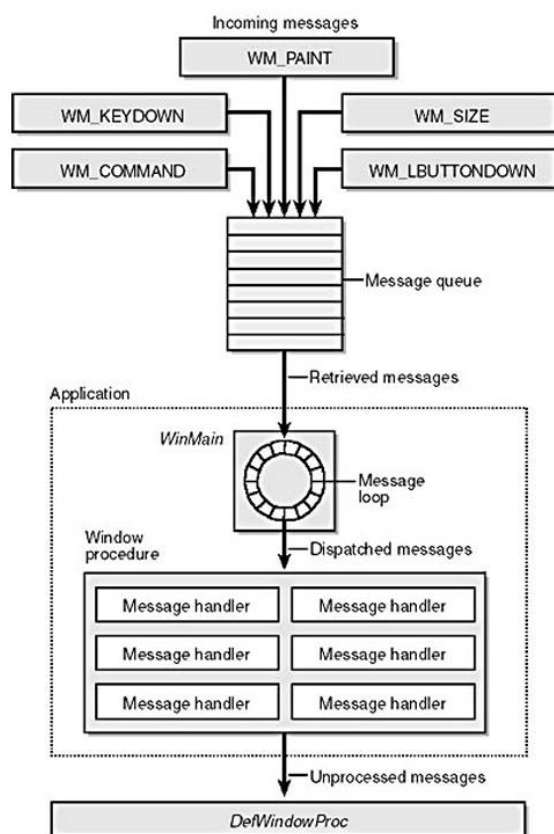
Još jedan C++ multiplatformski okvir vrijedan spomena je Qt koji pruža vrlo bogat skup biblioteka i moderno sučelje. U Qt-ju su napravljeni mnogi programi za uređivanje slika i medija, CAD te sučelja za embedded i mobilne sustave. [8]

3. FUNKCIONALNOST MFC-A

3.1. Windows programski model

Tradicionalni programi koriste proceduralni model izvođenja u kojem se naredbe izvršavaju redom od početka do kraja programa. Windows aplikacije koriste drugačiji pristup poznat kao event-driven programming model odnosno programski model temeljen na događajima. U takvom modelu aplikacija većinu vremena ne izvršava vlastiti programski kod, već čeka događaje koje generira operacijski sustav ili korisnik. Događaji mogu biti pritisak tipke na tipkovnici, klik mišem, promjena veličine prozora ili zahtjev za ponovnim iscrtavanjem sadržaja prozora. Kada se takav događaj dogodi, operacijski sustav generira odgovarajuću poruku i smješta je u red poruka (message queue). Program zatim preuzima poruke iz reda i prosljeđuje ih odgovarajućem prozoru na obradu. [2]

Slika 4 Windows programski model



(Programming Windows with MFC second edition, 1998)

Tablica 2 Najčešće poruke

Poruka	Događaj
WM_CHAR	Pritisnuta tipka ili je unesen znak na tipkovnici
WM_COMMAND	Šalje se kada korisnik pozove stavku naredbe iz izbornika ili kada kontrola pošalje obavijesnu poruku svom roditeljskom prozoru
WM_LBUTTONDOWN	Pritisnut je lijevi gumb miša
WM_LBUTTONUP	Pušten je lijevi gumb miša
WM_MOUSEMOVE	Pomaknut je miš
WM_CREATE	Prozor je napravljen
WM_DESTROY	Prozor je uništen
WM_SIZE	Prozor je promijenio veličinu
WM_PAINT	Prozor se treba ponovno nacrtati
WM_QUIT	Aplikacija se terminira

U klasičnom Win32 programiranju obrada poruka odvija se u funkciji nazvanoj Window Procedure (WndProc). Ta funkcija sadrži switch strukturu koja za svaku poruku određuje odgovarajuće ponašanje.

```
switch (message)
{
    case WM_PAINT:
        ...
        break;
    ...
    case WM_CLOSE:
        ...
        break;
}
```

Kod 1 switch naredba unutar WndProc()

3.2. Mapiranje poruka (Message Maps)

MFC mapiranje poruka je jedan od najvažnijih mehanizama u MFC-u. Omogućava automatsko mapiranje Windows poruka i naredbi na metode u klasama bez ručnog pozivanja RegisterClass() ili OnWndMsg().

```
// MyView.h
class CMyView : public CView {
    afx_msg void OnFileOpen();
    DECLARE_MESSAGE_MAP()
};

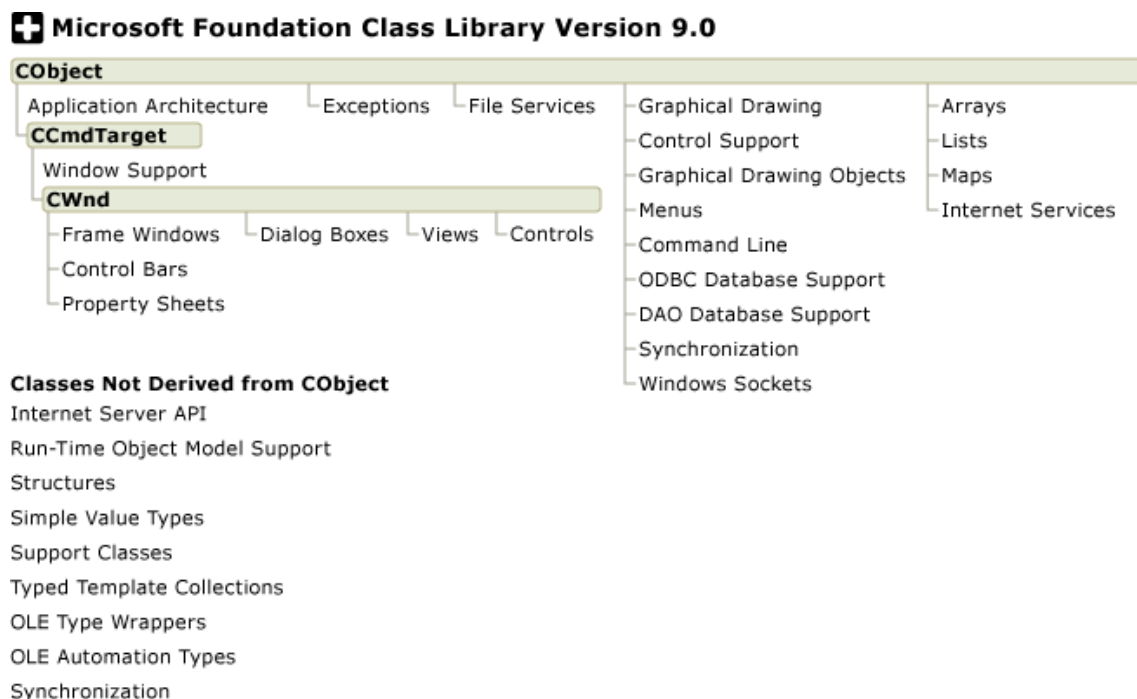
// MyView.cpp
BEGIN_MESSAGE_MAP(CMyView, CView)
    ON_COMMAND(ID_FILE_OPEN, &CMyView::OnFileOpen)
END_MESSAGE_MAP()

void CMyView::OnFileOpen() {}
```

Kod 2 mapiranje poruke, ovime se zamjenjuje switch u WndProc() prikazan u kodu 1

3.3. Hijerarhija klasa MFC-a

MFC je organiziran kao velika hijerarhija međusobno povezanih C++ klasa. Većina klasa unutar biblioteke izravno ili neizravno nasljeđuje klasu CObject, koja predstavlja temelj gotovo cijelog MFC-a.



Slika 5 Hijerarhijsko stablo CObject klase

Najvažnije funkcionalnosti klase CObject su podrška za serijalizaciju (serialization), Run-Time Class Information (RTCI) i podrška za dijagnostiku i otklanjanje pogrešaka.

Serijalizacija predstavlja postupak spremanja podataka objekta na trajni medij, primjerice datoteku na disku, te njihovo kasnije ponovno učitavanje. Primjer primjene serijalizacije uključuje:

- spremanje dokumenata
- spremanje postavki aplikacije
- spremanje konfiguracijskih podataka

Ova funkcionalnost posebno je važna u aplikacijama koje koriste dokument/pogled arhitekturu.

RTCI mehanizam omogućuje određivanje tipa objekta tijekom izvođenja programa. Primjerice, aplikacija može provjeriti pripada li neki objekt određenoj klasi bez potrebe za ručnim vođenjem informacija o tipu objekta. MFC je implementirao vlastiti sustav RTCI-ja prije nego što je standard C++ uveo mehanizam poznat kao Run-Time Type Information (RTTI).

CObject dodatno pomaže u otkrivanju curenja memorije (memory leaks). U debug konfiguraciji MFC prati dinamički alocirane objekte te može upozoriti programera u Debug Output prozoru Visual Studija ako neki objekt nije pravilno oslobođen prije završetka rada aplikacije.

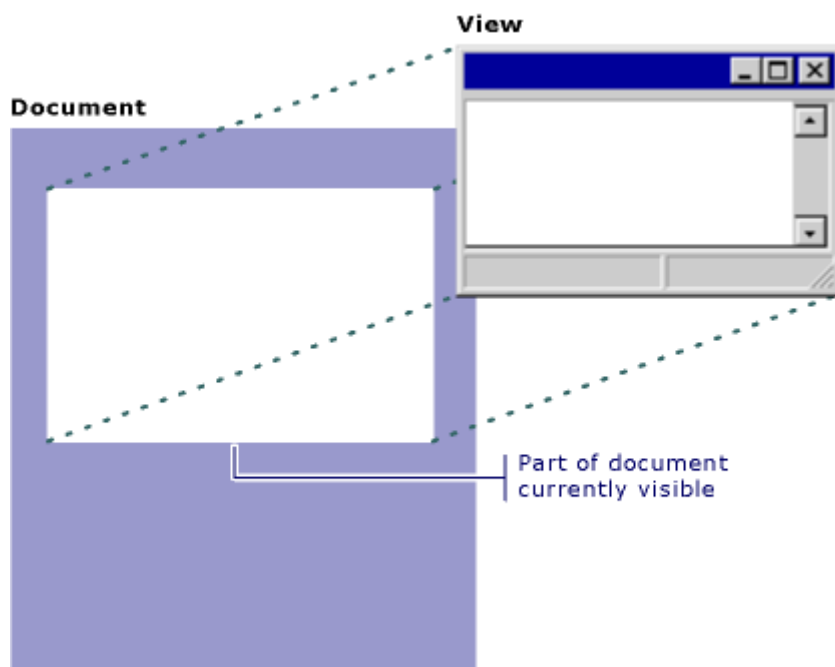
3.4. Dokument/pogled arhitektura

Jedna od najvažnijih sastavnica MFC aplikacijskog okvira je dokument/pogled arhitektura. Ovaj model predstavlja temelj mnogih MFC aplikacija jer omogućuje odvajanje podataka aplikacije od njihovog prikaza na korisničkom sučelju. MFC podršku za ovu arhitekturu implementira kroz klase CDocument i CView, dok klase poput CWinApp, CFrameWnd i drugih komponenti povezuju pojedine dijelove aplikacije u funkcionalnu cjelinu. Takvo odvajanje omogućuje da se isti podaci prikazuju na više različitih načina bez potrebe za dupliciranjem podataka. Na primjer, jedna aplikacija može imati više prikaza istog dokumenta, pri čemu svaka instanca klase CView prikazuje sadržaj na drugačiji način. Prednost ovog modela dolazi do izražaja kod složenijih aplikacija jer olakšava održavanje, proširivanje i organizaciju programskog koda. Programer može mijenjati način prikaza podataka bez promjene njihove interne reprezentacije te obratno.[2][6]

MFC aplikacijama koje koriste arhitekturu dokument/pogled pruža i dodatne mogućnosti koje bi u čistom Win32 programiranju zahtijevale znatno više programskog koda. Primjeri uključuju:

spremanje i učitavanje dokumenata, ispis i pregled ispisa (Print Preview), podršku za višestruke prikaze istog dokumenta te integraciju s OLE i Active Document tehnologijama.

Slika 6 Prikaz dokument/pogled veze



Izvor: [10]

3.5. AFX funkcije

Osim velikog broja C++ klasa, MFC sadrži i skup globalnih funkcija čiji nazivi započinju prefiksom Afx. Za razliku od članskih funkcija koje pripadaju određenim objektima, AFX funkcije mogu se pozivati iz bilo kojeg dijela programa bez potrebe za pristupom konkretnom MFC objektu. Ove funkcije predstavljaju dodatni sloj apstrakcije iznad Win32 API-ja te programeru omogućuju jednostavniji pristup funkcionalnostima okruženja.

Tablica 3 Najčešće AFX funkcije

Funkcija	Namjena
AfxMessageBox()	Prikaz dijaloškog okvira s porukom
AfxGetApp()	Dohvaća pointer (pokazivač) na objekt aplikacije
AfxGetAppName()	Dohvaća naziv aplikacije
AfxGetMainWnd()	Dohvaća pointer na glavni prozor aplikacije
AfxGetInstanceHandle()	Dohvaća handle trenutne instance aplikacije
AfxBeginThread()	Započinje thread
AfxEndThread()	Završava thread
AfxAbort()	Prekida izvođenje aplikacije

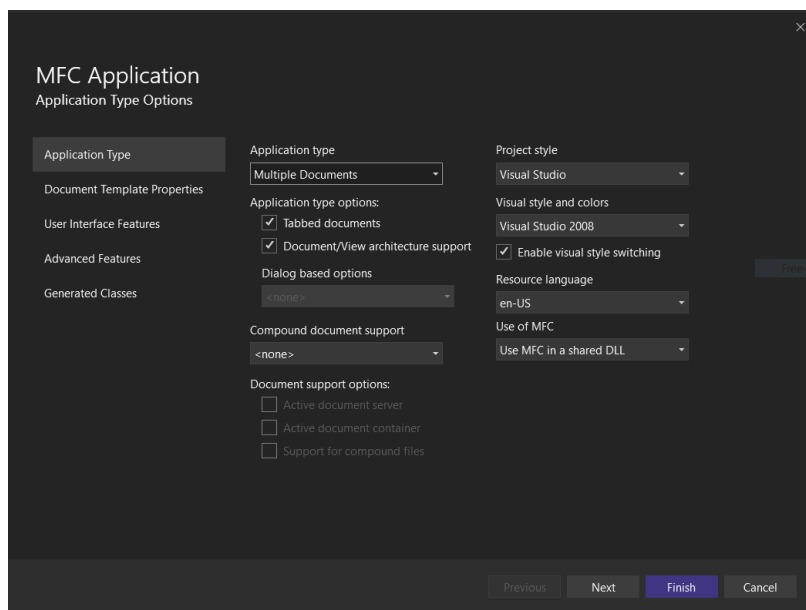
4. PODRŠKA U VISUAL STUDIJU

4.1. Integracija MFC-a u Visual Studio

MFC je od samih početaka usko povezan s Microsoftovim razvojnim okruženjem Visual C++. Programerima je omogućeno stvaranje, uređivanje, kompajliranje i otklanjanje pogrešaka MFC aplikacija unutar jedinstvenog razvojnog okruženja. Prilikom instalacije Visual Studija moguće je odabrati komponentu „C++ MFC for latest v143 build tools“ Desktop development with C++, koja uključuje MFC biblioteku, Visual C++ alatni lanac, dijagnostičke alate te predloške potrebne za razvoj MFC aplikacija. MFC je i dalje podržan u aktualnim verzijama Visual Studija, što omogućuje održavanje postojećih aplikacija, ali i razvoj novih Windows desktop programa temeljenih na ovoj tehnologiji.

4.2. MFC aplikacijski čarobnjak

Jedna od najvažnijih značajki Visual Studija za MFC je aplikacijski čarobnjak. Čarobnjak omogućuje odabir vrste aplikacije (SDI, MDI, dijaloška aplikacija), izgleda sučelja (Ribbon ili klasično), podrške za arhitekturu dokument/pogled te za ActiveX i COM.



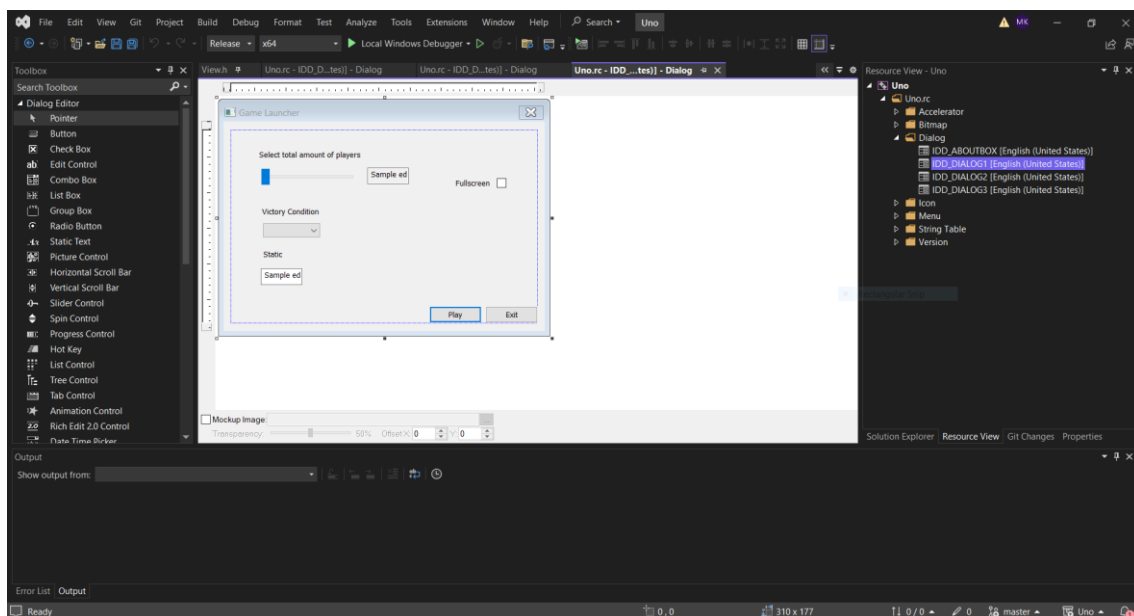
Slika 7 Aplikacijski čarobnjak

4.3. Uređivači resursa

Visual Studio uključuje Resource Editor, alat za vizualno oblikovanje korisničkog sučelja. Pomoću njega moguće je izrađivati:

- dijaloške okvire
- izbornike
- alatne trake
- Ribbon sučelja
- ikone
- pokazivače miša
- string tablice

Elementi se pohranjuju u resursne datoteke (.rc) koje Visual Studio automatski povezuje s aplikacijom.



Slika 8 Resource editor

Dvostrukim klikom na kontrolu Visual Studio automatski generira funkciju za obradu događaja (npr. `OnBnClickedButton1()`) i unos u mapu poruka..

5. PRAKTIČAN RAD – UNO KARTAŠKA IGRA

5.1. Svrha aplikacije

Aplikacija je računalna verzija poznate kartaške igre Uno.

Pravila igre:

- Igra se protiv 1 do 9 računalnih igrača.
- Prije početka igre se odabire broj igrača i trajanje igre, moguće je odabrati jednu partiju, igru do određenog broja pobjeda ili tradicionalno igranje na bodove.
- Na +2 i +4 je moguće odgovoriti s +2 odnosno +4, nakon čega će sljedeći igrač morati ili odgovoriti ili vući zbroj svih +2 ili +4 karata. Igrač koji je vukao zbog +2 ili +4 ne gubi potez i, ako ne može odigrati kartu, vuče još jednu.
- U igri s jednim protivnikom, nakon odigravanja karte za promjenu smjera ili preskakanje, igrač ponovno dolazi na red i mora vući kartu ako nema drugu kartu za odigrati, osim ako mu je to bila posljednja karta.



Slika 9 Sučelje aplikacije Uno

5.2. Struktura aplikacije

Aplikacija je izrađena korištenjem MFC biblioteke i programskog jezika C++. Za prikaz karata i korisničkog sučelja korišten je GDI+, dok su dijaloški okviri i standardne kontrole implementirani korištenjem MFC klasa.

Aplikacija je organizirana prema principima MFC okvira te koristi arhitekturu temeljenu na klasama CFrameWnd i CView. Za razliku od klasične MFC dokument/pogled aplikacije, ne koristi klasu CDocument, već vlastitu klasu Game koja služi kao model podataka i upravlja kompletnom logikom igre.

Klasa CUnoGameApp predstavlja ulaznu točku aplikacije i odgovorna je za:

- inicijalizaciju aplikacije,
- pokretanje GDI+ sustava,
- inicijalizaciju glavnog prozora,

Klasa View upravlja prikazom podataka i interakcijom s korisnikom.

Klasa Scoreboard služi za bilježenje bodova svakog igrača i nakon svake partije vraća informaciju ako je neki igrač pobijedio.

Struktura SavedGameSettings služi za spremanje podataka koje je korisnik unio pri početku igre.

Klasa Game predstavlja središnji dio aplikacije i sadrži kompletnu logiku igre:

- tijek igre,
- promjenu igrača,
- provjeru valjanosti poteza,
- obračun bodova,
- određivanje pobjednika.

Klasa Player predstavlja pojedinog igrača. Svaki igrač sadrži objekt 'Hand playerHand' koji upravlja kartama koje posjeduje.

Klasa Hand implementira funkcionalnosti:

- sortiranje karata,
- provjeru mogućih poteza,
- uklanjanje odigrane karte,

- provjeru pravila.

Klasa Deck predstavlja špil karata te sadrži `std::vector<Card>`

i metode za:

- miješanje karata,
- ponovno stvaranje špila,
- izvlačenje gornje karte.

Klasa PlayedCards predstavlja špil odigranih karata.

Klasa Card predstavlja osnovni podatkovni objekt igre. Definirana je pomoću enumeracija koje opisuju:

- boju karte,
- tip karte.

Svaka karta ima svoj ID koji se računa formulom: $\text{boja karte} \times 100 + \text{tip karte}$, ID se koristi za ID bitmape karte i na taj način karte dobivaju svoj izgled.

5.3. Korištene MFC komponente

Glavni prikaz implementiran je nasljeđivanjem klase:

```
class View : public CView
```

Ova klasa upravlja:

- prikazom karata,
- korisničkim unosom,
- osvježavanjem zaslona,
- komunikacijom s modelom igre.

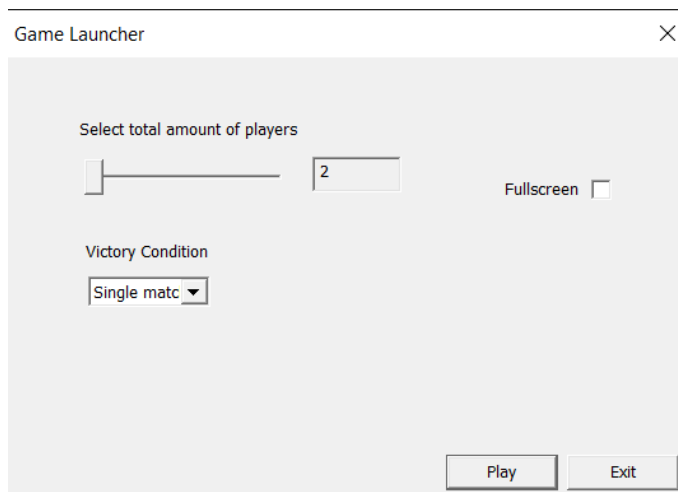
Za korisnički unos koriste se CButton, CDialogEx, CSliderCtrl, CComboBox, CEdit, CStatic, DDX_Control objekti, a za prikaz stanja svih igrača se koristi CListBox.

```
//View.h
class View : public CView
{
    ...
    CButton DrawButton;
    afx_msg void OnDrawButtonClick();
    ...
    DECLARE_MESSAGE_MAP()
}
```

```
};
//View.cpp
BEGIN_MESSAGE_MAP(View, CView)
    ...
    ON_BN_CLICKED(1001, &View::OnDrawButtonClick)
    ...
END_MESSAGE_MAP()
void View::OnDrawButtonClick(){...}
```

Kod 3 Primjer deklariranja CButton objekta te mapiranja poruke

Aplikacija koristi nekoliko dijaloških prozora, Jedan od njih je `GameSettingsDlg`, koji se prikazuje pri pokretanju aplikacije i u kojem korisnik odabire postavke igre.



Slika 10 Izgled GameSettings dijaloga

5.4. Korištenje GDI+ biblioteke

GDI+ (Graphics Device Interface Plus) predstavlja proširenje klasičnog Windows grafičkog sustava GDI, GDI+ omogućuje:

- prikaz bitmap slika,
- transparentnost (alpha channel),
- anti-aliasing,
- gradijent,
- transformacije i skaliranje slika,
- podršku za formate JPEG, PNG, BMP i GIF

GDI+ omogućuje kvalitetnije i jednostavnije crtanje u Win32 okruženju, ali se danas smatra zastarjelim zbog:

- većinu operacija izvodi korištenjem procesora (CPU) umjesto grafičke kartice (GPU),

- kod većeg broja animacija, efekata ili čestog ponovnog iscrtavanja prozora performanse mogu biti osjetno lošije od rješenja temeljenih na DirectX-u,
- ne nudi moderne animacije, hardverski ubrzano renderiranje, 3D prikaz.

```
//Uno.h
#include <gdiplus.h>
using namespace Gdiplus;
class CUnoGameApp : public CWinApp{
public:
    ULONG_PTR m_gdiplusToken;
}
//Uno.cpp
BOOL CUnoGameApp::InitInstance() {
    GdiplusStartupInput gdiplusStartupInput;
    GdiplusStartup(&m_gdiplusToken, &gdiplusStartupInput, NULL);
    ...
}
int CUnoGameApp::ExitInstance() {
    GdiplusShutdown(m_gdiplusToken);
    ...
    return CWinApp::ExitInstance();
}
//View.cpp
BOOL View::LoadImagesFromResource() {
    ...
    for (Card card : game.getPlayerhand()) {
        hand bitmaps.push_back(Gdiplus::Bitmap::FromResource(hModule,
MAKEINTRESOURCEW(card.Color*100+card.Type)));
    }
    ...
    return TRUE;
}
```

Kod 4 Primjer korištenja GDI+

ZAKLJUČAK

Promatrajući MFC iz današnje perspektive, može se zaključiti da se radi o tehnologiji koja se nalazi između dvije krajnosti. S jedne strane, MFC više nije suvremeno rješenje za razvoj novih desktop aplikacija. Mnogi problemi koje je MFC uspješno rješavao početkom 1990-ih danas imaju jednostavnija i produktivnija rješenja u tehnologijama poput WinUI-ja, WPF-a, Qt-a i wxWidgets-a. Nedostatak ugrađene podrške za podatkovno povezivanje, oslanjanje na mehanizam mapiranja poruka te veća količina programskog koda potrebna za izradu korisničkog sučelja čine razvoj sporijim u usporedbi s modernim okvirima.

S druge strane, MFC nije zastario u smislu da više nema praktičnu primjenu. Njegova povezanost s Win32 API-jem omogućuje vrlo dobre performanse i veliku razinu kontrole nad operacijskim sustavom. Upravo zbog toga i dalje postoji značajan broj aplikacija koje se oslanjaju na MFC ili izravno na Win32. Međutim, kod novih projekata kojima su performanse jednako važne kao produktivnost se često koriste alternativni okviri poput Qt-a ili wxWidgets-a, koji nude slične mogućnosti uz veću produktivnost i podršku za više platformi. Stoga se MFC danas može opisati kao tehnologija koja je izgubila status preporučenog izbora za nove projekte, ali još uvijek nije izgubila svoju vrijednost. Njegova budućnost vjerojatno neće biti obilježena velikim funkcionalnim nadogradnjama, već održavanjem postojećih sustava i postupnom migracijom prema novijim tehnologijama za čiji rok prelaska nije tehnološki problem, nego ograničen broj MFC programera. Ipak, ideja na kojoj je MFC izgrađen, odnosno pružanje jednostavnijeg i objektno orijentiranog pristupa razvoju Windows aplikacija, ostala je prisutna i u modernim Microsoftovim okvirima.

LITERATURA

1. Microsoft Learn, (2026), MFC Desktop Applications, <https://learn.microsoft.com/en-us/cpp/mfc/mfc-desktop-applications> (pristupljeno 29.9.2026.)
2. Jeff Prosise, (1999) *Programming Windows with MFC second edition*, Microsoft Press
3. Marian Luparu, (2017), Happy 25th Birthday MFC!, <https://devblogs.microsoft.com/cppblog/happy-25th-birthday-mfc/> (pristupljeno 30.9.2026.)
4. Marius Bancila (2015), Version history of VC++, MFC and ATL <https://mariusbancila.ro/blog/2015/08/12/version-history-of-vc-mfc-and-atl/> (pristupljeno 30.9.2026.)
5. Winmerge, <https://winmerge.org/> (pristupljeno 29.9.2026.)
6. Microsoft Learn, (2026), Windows Presentation Foundation Overview, <https://learn.microsoft.com/en-us/dotnet/desktop/wpf/overview/> (pristupljeno 29.9.2026.)
7. Microsoft Learn, (2026), WinUI 3, <https://learn.microsoft.com/en-us/windows/apps/winui/winui3/> (pristupljeno 29.9.2026.)
8. wxWidgets <https://wxwidgets.org/about/> (pristupljeno 30.9.2026.)
9. Shneor, Rotem (2012). *Handbook of Research on Born Globals, Chapter 10: Born Global Firms, Internet, and New Forms of Internationalization*. Edward Elgar. (pristupljeno 30.9.2026.)
10. Microsoft Learn, (2026), Document/View Architecture <https://learn.microsoft.com/en-us/cpp/mfc/document-view-architecture> (pristupljeno 29.9.2026.)
11. Microsoft Learn, (2020) Windows Internet <https://learn.microsoft.com/en-us/windows/win32/wininet/portal> (pristupljeno 30.9.2026.)

Izvor slike karata:

Wikimedia Commons, https://commons.wikimedia.org/wiki/File:UNO_cards_deck.svg

SAŽETAK

U ovom radu obrađena je biblioteka Microsoft Foundation Classes (MFC) koja predstavlja objektno-orijentirani okvir za razvoj Windows desktop aplikacija temeljenih na Win32 API-ju. U teorijskom dijelu rada prikazana je povijest razvoja MFC-a od njegovog nastanka početkom 1990-ih godina do današnjeg statusa tehnologije koja se primarno koristi za održavanje postojećih sustava. Također je provedena usporedba MFC-a s novijim Microsoftovim okvirima, poput WPF-a i WinUI-ja, te s nezavisnim rješenjima wxWidgets i Qt.

Nadalje su opisani osnovni koncepti i funkcionalnosti MFC-a, uključujući Windows programski model temeljen na događajima, mehanizmu mapiranja poruka, hijerarhiju klasa temeljenu na klasi CObject, dokument/pogled arhitekturu i AFX funkcije. Posebna pažnja posvećena je ulozi MFC-a kao aplikacijski okvir te njegovoj integraciji s razvojnim okruženjem Visual Studio kroz MFC aplikacijski čarobnjak i Resource Editor.

U praktičnom dijelu rada razvijena je desktop aplikacija koja implementira kartašku igru UNO korištenjem programskog jezika C++, MFC biblioteke i GDI+ grafičkog sustava. Analizirana je struktura aplikacije, korištene MFC komponente, organizacija programskog koda te primjena GDI+ biblioteke za prikaz grafičkih elemenata i korisničkog sučelja.

Ključne riječi: MFC, Win32 API, Visual Studio, C++, desktop aplikacije, Windows operacijski sustav

SUMMARY

This paper covers the Microsoft Foundation Classes (MFC) library, which is an object-oriented framework for developing Windows desktop applications based on the Win32 API. In the theoretical part of the paper, the history of MFC's development is presented, from its inception in the early 1990s to its current status as a technology primarily used for maintaining existing systems. A comparison of MFC with newer Microsoft frameworks, such as WPF and WinUI, and with independent solutions like wxWidgets and Qt was also conducted.

Furthermore, the basic concepts and functionalities of MFC are described, including the event-driven Windows programming model, the Message Maps mechanism, the class hierarchy based on the CObject class, the Document/View architecture, and AFX functions. Special attention is paid to the role of MFC as an application framework and its integration with the Visual Studio development environment through the MFC Application Wizard and Resource Editor.

In the practical part of the paper, a desktop application was developed that implements the card game UNO using the C++ programming language, the MFC library, and the GDI+ graphics system. The application's structure, the MFC components used, the organization of the program code, and the application of the GDI+ library for displaying graphical elements and the user interface were analyzed.

Keywords: MFC, Win32 API, Visual Studio, C++, desktop applications, Windows operating system